

Customer Feedback System – OpenAPI Stubs with Sample Payloads & Response Examples

This document provides concise OpenAPI (v3.0) stubs for each microservice along with sample request payloads and response examples. It also includes the deployment pipeline diagram for CI/CD.

Common Components

```
openapi: 3.0.3
info:
  title: Customer Feedback System APIs
  version: 0.2.0
servers:
  - url: https://api.example.com
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
  schemas:
    ProblemDetails:
      type: object
      properties:
        type: { type: string }
        title: { type: string }
        status: { type: integer }
        traceId: { type: string }
      errors:
        type: object
        additionalProperties:
          type: array
          items: { type: string }
security:
  - bearerAuth: []
```

Auth Service

```
openapi: 3.0.3
info: { title: Auth Service, version: 0.2.0 }
paths:
  /api/auth/register:
    post:
      summary: Register a new user
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password]
              properties:
                email: { type: string, format: email }
                password: { type: string, minLength: 8 }
                role: { type: string, enum: [Customer, StoreManager, Analyst, Admin] }
                storeId: { type: string, format: uuid }
            examples:
              sample:
                value:
                  email: manager@example.com
```

```

        password: "Str0ngP@ss!"
        role: StoreManager
        storeId: "2f1a8f7e-7e23-4e5d-8f12-8f7e23e58f12"
responses:
  '201':
    description: Created
    content:
      application/json:
        examples:
          created:
            value:
              id: "b3f2a0c7-1c2b-4bf6-9e21-5f3d4b1a2c9e"
              email: manager@example.com
              role: StoreManager
  '400':
    description: Validation error
    content:
      application/json:
        schema: { $ref: '#/components/schemas/ProblemDetails' }
        examples:
          invalid:
            value:
              title: Validation Error
              status: 400
              errors:
                password: ["Password must be at least 8 characters."]
/api/auth/login:
  post:
    summary: Login
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [email, password]
            properties:
              email: { type: string, format: email }
              password: { type: string }
          examples:
            sample:
              value:
                email: manager@example.com
                password: "Str0ngP@ss!"
    responses:
      '200':
        description: Tokens
        content:
          application/json:
            schema:
              type: object
              properties:
                accessToken: { type: string }
                refreshToken: { type: string }
            examples:
              ok:
                value:
                  accessToken: "eyJhbGciOi..."
                  refreshToken: "def50200..."
      '401': { description: Unauthorized }
/api/auth/refresh:
  post:
    summary: Refresh token
    requestBody:
      required: true
      content:
        application/json:
          schema:

```

```

      type: object
      required: [refreshToken]
      properties: { refreshToken: { type: string } }
    examples:
      sample:
        value:
          refreshToken: "def50200..."
  responses:
    '200':
      description: New tokens
      content:
        application/json:
          examples:
            ok:
              value:
                accessToken: "eyJhbGciOi..."
                refreshToken: "def50201..."
    '401': { description: Unauthorized }
/api/auth/me:
  get:
    summary: Get current user
    responses:
      '200':
        description: User profile
        content:
          application/json:
            examples:
              ok:
                value:
                  id: "b3f2a0c7-1c2b-4bf6-9e21-5f3d4b1a2c9e"
                  email: manager@example.com
                  roles: ["StoreManager"]
      '401': { description: Unauthorized }

```

Feedback Service

```

openapi: 3.0.3
info: { title: Feedback Service, version: 0.2.0 }
paths:
  /api/feedback:
    post:
      summary: Submit feedback
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [storeId, rating]
              properties:
                storeId: { type: string, format: uuid }
                customerId: { type: string, format: uuid }
                rating: { type: integer, minimum: 1, maximum: 5 }
                comment: { type: string, maxLength: 2000 }
                categoryId: { type: string, format: uuid }
                metadata: { type: object }
            examples:
              sample:
                value:
                  storeId: "a9d3d7fd-6f47-4f3a-9a3d-3d7fd6f479a3"
                  customerId: "c1a2b3c4-d5e6-7890-abcd-ef1234567890"
                  rating: 4
                  comment: "Staff were helpful but checkout was slow."
                  categoryId: "0d9f1c0a-9b6e-4b3e-8b3c-7f1c0a9b6e4b"
                  metadata:
                    device: "web"

```

```

        locale: "en-IN"
responses:
  '201':
    description: Created
    content:
      application/json:
        examples:
          created:
            value:
              id: "9f4a1d3c-7b8e-4f0e-9a3c-1d3c7b8e4f0e"
              createdAt: "2025-12-08T08:44:12Z"
  '400':
    description: Validation error
    content:
      application/json:
        examples:
          invalid:
            value:
              title: "Validation Error"
              status: 400
              errors:
                rating: ["Rating must be between 1 and 5."]
get:
  summary: List feedback (paginated)
  parameters:
    - in: query
      name: storeId
      schema: { type: string, format: uuid }
    - in: query
      name: page
      schema: { type: integer, minimum: 1, default: 1 }
    - in: query
      name: size
      schema: { type: integer, minimum: 1, maximum: 100, default: 20 }
    - in: query
      name: from
      schema: { type: string, format: date-time }
    - in: query
      name: to
      schema: { type: string, format: date-time }
  responses:
    '200':
      description: Paged list
      content:
        application/json:
          examples:
            ok:
              value:
                pageInfo:
                  page: 1
                  size: 20
                  totalItems: 135
                items:
                  - id: "9f4a1d3c-7b8e-4f0e-9a3c-1d3c7b8e4f0e"
                    storeId: "a9d3d7fd-6f47-4f3a-9a3d-3d7fd6f479a3"
                    rating: 4
                    comment: "Helpful staff; slow checkout."
                    createdAt: "2025-12-08T08:44:12Z"
/api/feedback/{id}:
get:
  summary: Get feedback by id
  parameters:
    - in: path
      name: id
      required: true
      schema: { type: string, format: uuid }
  responses:
    '200':

```

```

    description: Feedback
    content:
      application/json:
        examples:
          ok:
            value:
              id: "9f4a1d3c-7b8e-4f0e-9a3c-1d3c7b8e4f0e"
              storeId: "a9d3d7fd-6f47-4f3a-9a3d-3d7fd6f479a3"
              rating: 4
              comment: "Helpful staff; slow checkout."
              categoryId: "0d9f1c0a-9b6e-4b3e-8b3c-7f1c0a9b6e4b"
              createdAt: "2025-12-08T08:44:12Z"
    '404': { description: Not found }
  put:
    summary: Update feedback
    requestBody:
      required: true
      content:
        application/json:
          examples:
            sample:
              value:
                rating: 5
                comment: "Team resolved the issue promptly."
    responses:
      '200':
        description: Updated
        content:
          application/json:
            examples:
              ok:
                value:
                  id: "9f4a1d3c-7b8e-4f0e-9a3c-1d3c7b8e4f0e"
                  rating: 5
                  updatedAt: "2025-12-08T09:10:00Z"
      '403': { description: Forbidden }
  delete:
    summary: Delete feedback (soft)
    responses:
      '204': { description: Deleted }

```

Reporting Service

```

openapi: 3.0.3
info: { title: Reporting Service, version: 0.2.0 }
paths:
  /api/reporting/summary:
    get:
      summary: Store manager summary
      parameters:
        - in: query
          name: storeId
          required: true
          schema: { type: string, format: uuid }
        - in: query
          name: from
          schema: { type: string, format: date-time }
        - in: query
          name: to
          schema: { type: string, format: date-time }
      responses:
        '200':
          description: KPIs and charts data
          content:
            application/json:
              examples:

```

```

      ok:
        value:
          kpis:
            totalFeedback: 245
            avgRating: 4.2
            positiveCount: 180
            negativeCount: 35
          trends:
            - day: "2025-12-01"
              count: 8
              avgRating: 4.1
            - day: "2025-12-02"
              count: 12
              avgRating: 4.3
/api/reporting/distribution:
  get:
    summary: Ratings histogram
    parameters:
      - in: query
        name: storeId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Histogram bins
        content:
          application/json:
            examples:
              ok:
                value:
                  bins:
                    - rating: 1
                      count: 5
                    - rating: 2
                      count: 12
                    - rating: 3
                      count: 38
                    - rating: 4
                      count: 102
                    - rating: 5
                      count: 88

```

Catalog Service

```

openapi: 3.0.3
info: { title: Catalog Service, version: 0.2.0 }
paths:
  /api/categories:
    get:
      summary: List categories
      parameters:
        - in: query
          name: storeId
          schema: { type: string, format: uuid }
      responses:
        '200':
          description: Categories
          content:
            application/json:
              examples:
                ok:
                  value:
                    items:
                      - id: "a1b2c3d4-1111-2222-3333-444455556666"
                        name: "Checkout"
                      - id: "b2c3d4e5-7777-8888-9999-0000aaaa1111"

```

```

        name: "Product Quality"
post:
  summary: Create category
  requestBody:
    required: true
    content:
      application/json:
        examples:
          sample:
            value:
              name: "Customer Service"
              description: "Interactions with staff"
  responses:
    '201':
      description: Created
      content:
        application/json:
          examples:
            created:
              value:
                id: "c3d4e5f6-1234-5678-90ab-cdef12345678"
                name: "Customer Service"

```

Analytics Service

```

openapi: 3.0.3
info: { title: Analytics Service, version: 0.2.0 }
paths:
  /api/analytics/feedback/{id}:
    get:
      summary: Get analytics for feedback
      parameters:
        - in: path
          name: id
          required: true
          schema: { type: string, format: uuid }
      responses:
        '200':
          description: Sentiment and category
          content:
            application/json:
              examples:
                ok:
                  value:
                    feedbackId: "9f4a1d3c-7b8e-4f0e-9a3c-1d3c7b8e4f0e"
                    sentimentScore: 0.73
                    sentimentLabel: "Positive"
                    categorySuggestion:
                      id: "0d9f1c0a-9b6e-4b3e-8b3c-7f1c0a9b6e4b"
                      name: "Checkout"
        '404': { description: Not found }
  /api/analytics/reprocess/{id}:
    post:
      summary: Reprocess a single feedback item
      parameters:
        - in: path
          name: id
          required: true
          schema: { type: string, format: uuid }
      responses:
        '202':
          description: Accepted
          content:
            application/json:
              examples:
                accepted:

```

```
value:  
  jobId: "job-20251208-000123"  
  status: "Queued"
```


Deployment Pipeline Diagram

Pipeline diagram image not found; please generate cicd_flow.png.